

Installing the Anthropic SDK and API keys

Claude Certified Developer, Foundation (CCDV-F) · UST masterclass

EN

Session 7 of 12 · English notes

Session 7 (17 August 2026) moved from GUI products into **calling Claude from code**. Vijay's lab used Python notebooks: install the Anthropic SDK, load a secret from `.env`, construct a client, then call `messages.create`. The exam cares about terminology (SDK versus REST), where the key lives, and which method you invoke — not about memorising a notebook cell order.

Two ways onto the same Messages API

Anthropic exposes Claude as HTTP. You can POST to the REST Messages endpoint yourself (URL, headers, JSON body, bearer/API key). The **Anthropic SDK** is a language-specific wrapper around that REST surface. In Python it is the package you install; in Java it would be an SDK artefact. A cohort question put it plainly: in Python people say “library”; the exam wording is **SDK (software development kit)**. Same idea: typed helpers, retries, streaming helpers, and a client object so you do not hand-roll every request.

Path	What you own	When you use it
REST	HTTP client, auth header, JSON schema, error mapping	Non-Python stacks, gateways, or when you must see the raw wire format
SDK	Install package, create client, call methods	Class default; production Python/TypeScript apps
Agent SDK (later)	Orchestration on top of messages/tools	Agents — not this chapter's client

ANTHROPIC SDK IS NOT AGENT SDK

Vijay distinguished **Anthropic SDK** (conversations / Messages API) from **Agent SDK** (orchestration). Tools, MCP, and agents return in later modules. Foundation questions in this week are about `client.messages.create`, not `LangChain`.

Install, environment, and the client object

The lab pattern was: install dependencies in the notebook (often with Jupyter `%%capture` so pip noise does not flood the cell), load environment variables, instantiate `anthropic.Anthropic()` (or the equivalent client constructor in the current SDK). That client holds nested resources. Under **messages** you saw at least `create` (one-shot / non-streaming by default), `stream`, and **batches**. Dot notation is how you reach them: `client.messages.create(...)`.

- Treat the first cells of Vijay's notebooks as boilerplate: SDK install, `load_dotenv()`, client, default model id.
- The GitHub repo for the class holds notebooks; you still create your own `.env` with the key. Never commit that file.

- First call after a cold start can feel slow; that is network + model warmup, not a broken install.

API keys: console, .env, and what the SDK injects

Authentication is a **secret API key** issued on the Anthropic / Claude API platform (console), not the Chat or Claude Code Pro subscription. Vijay stored it in `.env` and loaded it with Python's `dotenv` helper so the process environment contained the key. The SDK then attaches it on each request. You *can* pass the key explicitly into the client constructor; you should not paste it into every cell. That is the point of a development kit: keep secrets out of source.

Classroom Q&A that maps to the exam

- You do not have to pass the key as a parameter of `create` if the client already loaded `ANTHROPIC_API_KEY` (name as in current SDK docs).
- Claude Pro / Claude Code monthly plans are **not** the same billing model as API credits. API work is console keys and token pricing.
- Enterprise trainers may not be able to screen-share key generation. Learn the console flow from official docs, not from a missing screenshot.
- First-time API accounts were described as coming with a small free credit (Vijay said on the order of \$5); treat amounts as perishable marketing, not exam gospel.

CONFIDENTIALITY

Do not put UST client secrets into a personal API key, a shared notebook, or a public GitHub repo. Rotate a key if it was pasted in chat or committed.

The create call: model, max_tokens, messages

A minimal invocation needs three ideas even if the SDK has defaults: **which model** (model id string), **max_tokens** (upper bound on *generated* tokens — Vijay's demos used 1024 as the working default), and **messages** (a list of role/content objects). Content is not “just a string prompt”: each item has a **role** (user, later assistant) and content (text in the Hello Claude demo). In an application, user content is the live user input substituted into that structure.

The return value is a **Message** object (dict-like in Python). You typically walk `message.content`, take the first block, and read `.text` on a text block. Printing the whole object is useful in class because you also see `id`, `model`, `role` (assistant), `stop_reason`, `usage`, and later fields such as `container` (cache / routing — module 09).

What a successful Message looks like

- **id** — unique message id from the API.

- **content** — list of blocks. Text answers arrive as a **text** block with a `text` property. Citations arrays appear when you enable citations (module 09 / RAG).
- **role** — assistant for model output.
- **stop_reason / stop_details** — why generation ended. If you did not hit the output cap, you often see a clean end-of-turn (class language: completed / end turn). If `max_tokens` cuts the answer, stop details explain the abrupt stop.
- **usage** — input vs output (and later cache) token counts for that request.

REST body versus SDK parameters

Because the platform is still REST, every SDK call is still: Anthropic URL + JSON body. The body includes model, messages, system (if any), temperature, `max_tokens`, stream flag, tools, thinking, cache control, and so on. **The API is stateless.** Repeating a system prompt on every request is not a bug; it is how HTTP APIs work. Prompt caching (module 09) is the supported way to avoid re-paying for a huge stable prefix.

DOCS OVER THE LIVE NOTEBOOK UI

Package names, constructor signatures, and Jupyter magic can change. Trust **Messages API** and the current Python SDK README if a class screenshot disagrees.

Study checklist

1. Name the two access paths: REST and SDK; SDK wraps REST.
2. State where the key is created (API console) and how the class loaded it (`.env` + `dotenv`).
3. List messages methods mentioned in class: create, stream, batches.
4. Name the three core create inputs: model, `max_tokens`, messages.
5. Extract assistant text from `content[0].text` (or equivalent) without dumping the whole object in production.

Official documentation

• Messages API

<https://docs.claude.com/en/api/messages>

Canonical request and response fields for create.

• API overview

<https://docs.claude.com/en/api/overview>

Auth, clients, and how the HTTP API is organised.

• Workspaces

<https://docs.claude.com/en/docs/build-with-claude/workspaces>

Where organisation keys and projects are grouped in console.

• Rate limits

<https://docs.claude.com/en/api/rate-limits>

Why a first demo can fail even with a valid key.

- **Where to find API documentation**

<https://support.claude.com/en/articles/8114490-where-can-i-find-your-api-documentation>

Support article pointing at docs.claude.com.

Study notes distilled from the UST CCD Foundation masterclass. Prefer Anthropic's current docs over session anecdotes. Not official exam material.